

07 - zz - Spine Leaf Architecture - EXERCISE (my solution)

Important

Decision: I will not do the excersis right now, because it goes much to deep into the networking aspect right now for my level of knowledge. (It would be no problem to solve, but much to time consuming with a benefit that is not high enough for me right now)

Lesson 5 Exercises: Spine-Leaf BGP Fabric

Complete these exercises to understand how Clos spine-leaf fabrics provide scalable, resilient data center connectivity using eBGP.

Exercise 1: Deploy and Configure the Fabric

Objective: Deploy the Clos fabric, configure eBGP on all devices, and verify end-to-end connectivity.

Steps

1. Deploy the topology:

```
cd lessons/clab/05-spine-leaf-bgp
containerlab deploy -t topology/lab.clab.yml
```

 done

1. Apply BGP config to all 6 routers using gNMIc (run from the `gnmic/` directory so `.gnmic.yml` provides credentials):

```
cd gnmic
gnmic -a clab-spine-leaf-bgp-spine1:57400 set --request-file
configs/spine1-bgp.json
gnmic -a clab-spine-leaf-bgp-spine2:57400 set --request-file
configs/spine2-bgp.json
gnmic -a clab-spine-leaf-bgp-leaf1:57400 set --request-file
configs/leaf1-bgp.json
gnmic -a clab-spine-leaf-bgp-leaf2:57400 set --request-file
configs/leaf2-bgp.json
gnmic -a clab-spine-leaf-bgp-leaf3:57400 set --request-file
configs/leaf3-bgp.json
gnmic -a clab-spine-leaf-bgp-leaf4:57400 set --request-file
configs/leaf4-bgp.json
```

✓ done

1. Before verifying, examine what the config files applied. Open `gnmic/configs/leaf1-bgp.json` and identify:
 - **Routing policies:** `import-all`, `export-connected`, and `export-bgp` -- why does a default-deny NOS like SR Linux need all three?
 - **Prefix-set filter:** `host-subnets` on `export-connected` -- what does `10.20.0.0/16` `mask-length-range 24..24` match, and what does it exclude?
 - **Multipath:** `maximum-paths: 2` under `afi-safi ipv4-unicast` -- what is the SR Linux default, and why does ECMP require changing it?
 - **Shared spine ASN:** Why do both spine neighbors have the same peer-as (65000)? What does RFC 7938 say about spine ASN assignment?

✓ done

1. Verify all 8 BGP sessions are established:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli -c "show network-
instance default protocols bgp neighbor"
```

Check all 6 devices -- each leaf should have 2 sessions (one per spine), each spine should have 4 sessions (one per leaf).

2. Verify end-to-end connectivity with cross-leaf pings:

```
docker exec clab-spine-leaf-bgp-host1 ping -c 3 10.20.2.2 # host1 ->
host2
docker exec clab-spine-leaf-bgp-host1 ping -c 3 10.20.3.2 # host1 ->
host3
docker exec clab-spine-leaf-bgp-host1 ping -c 3 10.20.4.2 # host1 ->
host4
docker exec clab-spine-leaf-bgp-host2 ping -c 3 10.20.4.2 # host2 ->
host4
```

Deliverables

- BGP neighbor output from a leaf (showing 2 established sessions)
- BGP neighbor output from a spine (showing 4 established sessions)

Exercise 2: Read the Fabric Routing Table -- Observe ECMP

Objective: Understand equal-cost multipath (ECMP) in a spine-leaf fabric and the configuration required to enable it.

Steps

1. Verify that BGP multipath is configured on leaf1:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli -c "info network-
instance default protocols bgp afi-safi ipv4-unicast multipath"
```

You should see `maximum-paths 2`. SR Linux defaults to `maximum-paths 1` (single best path only). Without this setting, BGP picks one spine and ignores the other -- no ECMP.

2. Check leaf1's routing table:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli -c "show network-
instance default route-table ipv4-unicast summary"
```

- #### 3. Look for routes to other leaves' host subnets (10.20.2.0/24, 10.20.3.0/24, 10.20.4.0/24). Each should show 2 next-hops (ECMP) -- one via each spine. The summary at the bottom should show `IPv4 prefixes with active ECMP routes: 3`.

4. Verify that only host subnets appear in BGP -- no /31 fabric link prefixes. The `host-subnets` prefix-set on `export-connected` filters these out. Compare leaf1's BGP routes to its local routes to confirm.
5. Traceroute from host1 to host4 -- run it multiple times:

```
docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
```

The path is always 4 hops: host -> leaf1 -> spine -> leaf4 -> host4. But which spine may vary between runs.

6. Answer these questions:
 - How many hops between any two hosts? (Always 4: host-leaf-spine-leaf-host)
 - How many equal-cost paths exist between any two leaves? (2, one per spine)
 - What happens to bandwidth if you add a third spine? (50% more aggregate bandwidth, 3 ECMP paths)

Deliverables

- Routing table showing ECMP entries (2 next-hops per remote leaf subnet)
- Traceroute output from multiple runs
- Answers to the 3 questions above

Exercise 3: Break/Fix -- Spine Failure (Fabric Resilience)

Objective: Observe that a spine-leaf fabric survives spine failures with reduced capacity but no connectivity loss.

Setup (break it)

```
docker exec -it clab-spine-leaf-bgp-spine1 sr_cli
```

Inside SR Linux -- disable all interfaces:

```
enter candidate
set / interface ethernet-1/1 admin-state disable
set / interface ethernet-1/2 admin-state disable
set / interface ethernet-1/3 admin-state disable
set / interface ethernet-1/4 admin-state disable
commit now
exit
```

Symptom

```
# Brief connectivity disruption, then full recovery through spine2
docker exec clab-spine-leaf-bgp-host1 ping -c 10 -i 1 10.20.4.2
```

Your Task

1. Run continuous pings between hosts on different leaves:

```
docker exec clab-spine-leaf-bgp-host1 ping -c 30 -i 1 10.20.4.2
```

2. Check leaf1's routing table -- ECMP entries should reduce from 2 paths to 1:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli -c "show network-
instance default route-table ipv4-unicast summary"
```

3. Check BGP neighbors on leaf1 -- the spine1 session should be down:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli -c "show network-
instance default protocols bgp neighbor"
```

4. Traceroute -- all traffic now goes through spine2:

```
docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
```

5. Fix -- re-enable spine1:

```
docker exec -it clab-spine-leaf-bgp-spine1 sr_cli
```

Inside SR Linux:

```
enter candidate
set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/2 admin-state enable
set / interface ethernet-1/3 admin-state enable
set / interface ethernet-1/4 admin-state enable
commit now
exit
```

6. Verify ECMP paths return to 2:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli -c "show network-  
instance default route-table ipv4-unicast summary"
```

Deliverables

- Ping output showing brief loss then recovery
 - Routing table before/after showing ECMP path count change (2 -> 1 -> 2)
 - Explanation of fabric resilience: why losing a spine degrades capacity but not connectivity
-

Exercise 4 (Challenge): Break/Fix -- Route Leak

Objective: Observe how a more-specific prefix can hijack traffic in a fabric.

Setup (break it)

On leaf1, add a more-specific route that covers leaf4's host subnet:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli
```

Inside SR Linux:

```

enter candidate
set / routing-policy prefix-set hijack prefix 10.20.4.0/25 mask-length-range
exact
set / routing-policy policy export-hijack statement 10 match prefix-set
hijack
set / routing-policy policy export-hijack statement 10 action policy-result
accept
set / routing-policy policy export-hijack statement 20 match protocol local
set / routing-policy policy export-hijack statement 20 action policy-result
accept
set / routing-policy policy export-hijack default-action policy-result
reject
set / network-instance default protocols bgp group spines export-policy
[export-hijack]
set / network-instance default static-routes route 10.20.4.0/25 admin-state
enable
set / network-instance default next-hop-groups group nhg-blackhole admin-
state enable
set / network-instance default next-hop-groups group nhg-blackhole nexthop 1
ip-address 192.0.2.1
set / network-instance default static-routes route 10.20.4.0/25 next-hop-
group nhg-blackhole
commit now
exit

```

Symptom

```

# host2 and host3 CANNOT reach host4
docker exec clab-spine-leaf-bgp-host2 ping -c 3 -W 5 10.20.4.2
docker exec clab-spine-leaf-bgp-host3 ping -c 3 -W 5 10.20.4.2
]
# Traffic to host4 gets sucked into leaf1's blackhole route

```

Your Task

1. Check routing tables on leaf2 and leaf3 -- look for a /25 route that should not be there:

```

docker exec -it clab-spine-leaf-bgp-leaf2 sr_cli -c "show network-
instance default route-table ipv4-unicast summary"
docker exec -it clab-spine-leaf-bgp-leaf3 sr_cli -c "show network-
instance default route-table ipv4-unicast summary"

```

2. Traceroute from host2 to host4 -- traffic goes to leaf1 (the hijacker) and dies:

```

docker exec clab-spine-leaf-bgp-host2 traceroute -n -w 2 10.20.4.2

```

3. Identify the rogue /25 prefix and trace it back to leaf1.

4. Fix -- remove the hijack config on leaf1:

```
docker exec -it clab-spine-leaf-bgp-leaf1 sr_cli
```

Inside SR Linux:

```
enter candidate
delete / routing-policy prefix-set hijack
delete / routing-policy policy export-hijack
delete / network-instance default static-routes route 10.20.4.0/25
delete / network-instance default next-hop-groups group nhg-blackhole
set / network-instance default protocols bgp group spines export-policy
[export-connected export-bgp]
commit now
exit
```

5. Verify host4 is reachable again:

```
docker exec clab-spine-leaf-bgp-host2 ping -c 3 10.20.4.2
docker exec clab-spine-leaf-bgp-host3 ping -c 3 10.20.4.2
```

Deliverables

- Routing table showing the rogue /25 prefix
- Explanation of longest-prefix-match hijacking: why a /25 wins over a /24 even though the /24 is the legitimate route

Cleanup

After completing all exercises:

```
# Destroy the lab
containerlab destroy -t topology/lab.clab.yml --cleanup
# Verify
docker ps | grep clab
```


Validation

Run the automated tests:

```
pytest tests/ -v
```

Completion Checklist

- ☐ Exercise 1: Deployed fabric, configured eBGP on all 6 routers, verified cross-leaf connectivity
- ☐ Exercise 2: Read routing tables, observed ECMP paths, ran traceroutes
- ☐ Exercise 3: Simulated spine failure, observed resilience and ECMP path reduction
- ☐ Exercise 4: Diagnosed and fixed route leak (longest-prefix-match hijack)

Next Steps

Commit your exercises to your fork and proceed to Lesson 6 (coming soon).